# Design Pattern Identification and Evolution in a Django based Digital Document Management Application

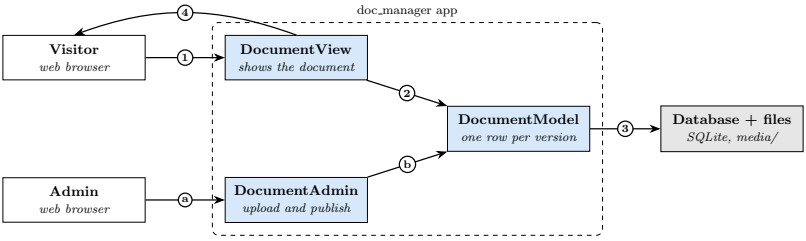
Team B17 — Raaghavan M S (AM.SC.U4CSE23330), V Sriman Vashishta (AM.SC.U4CSE23368), CSE-D

23CSE455 Design Patterns — Case Study

2026-10-07

# The app: `impicode/doc_manager` 1.0.2

- A very small real Django app (MIT, 138 lines of code, 11 tests).
- An admin uploads versions of a document, e.g. a Privacy Policy (PDF or HTML).
- Old versions are kept; exactly one version is *published* for visitors.

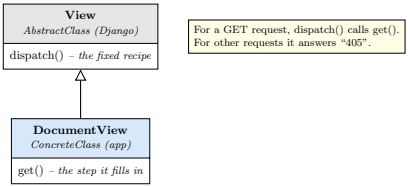



**Visitor:** **1** opens the page → **2** the view asks for the published version → **3** its row and file are read → **4** the file is sent back
**Admin:** **a** uploads a version or clicks "Publish" → **b** the file is checked and saved, or this version becomes the published one

# The two patterns, in plain words

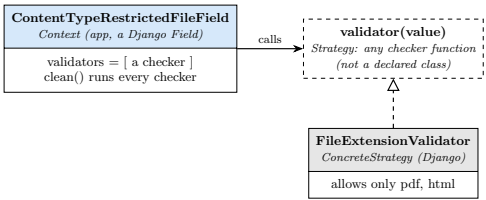
## Template Method

A parent class owns a fixed *recipe*.
A child class fills in one step.




## Strategy

An object keeps a *replaceable helper*
and calls it the same way every time.



# Part 1 — what we found: 2 types, 4 instances

| Instance | Pattern | Django part / app part |
|---|---|---|
| Published Document Page | Template Method | `View.dispatch` → `DocumentView.get` |
| Admin Read-only Fields | Template Method | `ModelAdmin` recipe → `get_readonly_fields` |
| Upload File Checks | Strategy | validators ← `FileExtensionValidator` |
| Upload Widget Choice | Strategy | widget ← `FileInput` |

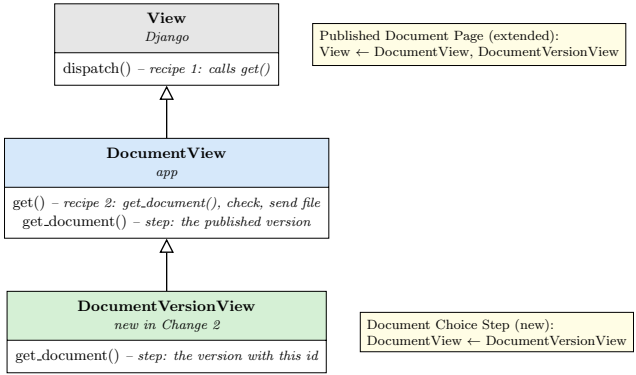
- Every instance is proven by a probe test that runs the real code (10/10).
- 7 candidates rejected (e.g. Singleton, Decorator, Factory Method), 0 uncertain.

# Part 2 — three changes, each with a small refactoring first

| Change | Refactoring | Pattern effect | Tests |
|---|---|---|---|
| CH01 accept .txt files | Extract Constant: one `FILE_TYPES` table | 2 Strategy + 1 TM *modified* | 11 → 19 |
| CH02 open an older version | Extract Method `get_document()` | new TM "Document Choice Step" | 19 → 22 |
| CH03 lock a published file | Rename Parameter | admin hook *modified* | 22 → 25 |

- New tests failed before each change and passed after; all logs are saved.
- Instances: 4 → 4 → 5 → 5. LOC 138 → 158. Highest CC 5 → 4.

# Part 2 — a new Template Method appeared (CH02)




The new class changes only *one step*: which version to show (drafts stay hidden).

# Part 3 — same problems in four languages

| | Template Method | | | Strategy | | |
|---|---|---|---|---|---|---|
| Language | LOC | CC | ms | LOC | CC | ms |
| Python | 20 | 3 | 73.2 | 31 | 5 | 79.1 |
| JavaScript | 28 | 3 | 80.2 | 40 | 5 | 78.7 |
| Java | 30 | 3 | 201.8 | 51 | 5 | 224.7 |
| C++ | 38 | 3 | 13.4 | 63 | 5 | 12.5 |

- All 8 programs pass the same shared cases (7/7 and 8/8).
- Java and C++ catch a missing step at compile time; Python and JavaScript at run time.

# LLM use (Claude Opus 5.5) — checked claim by claim

| Part | Correct | Partial | Wrong | What we used |
|---|---|---|---|---|
| 1 discovery | 11 | 1 | 2 | a missed instance (proven first) |
| 2 evolution | 11 | 3 | 1 | warning: CH02 showed drafts $\rightarrow$ fixed |
| 3 languages | 8 | 1 | 0 | generated code tested only (8/8 pass) |

- Prompts, inputs, raw outputs and our checks are saved for every run.

# Findings

- Even a tiny Django app uses Template Method and Strategy: Django gives the fixed part, the app fills in the rest.
- The patterns made each change small: one table entry, one step, one hook.
- Refactoring first, with tests before and after, kept every change safe.
- Bugs found: the page crashed for other file types and the `accept` hint was ignored (both fixed); the admin search crashes (documented).